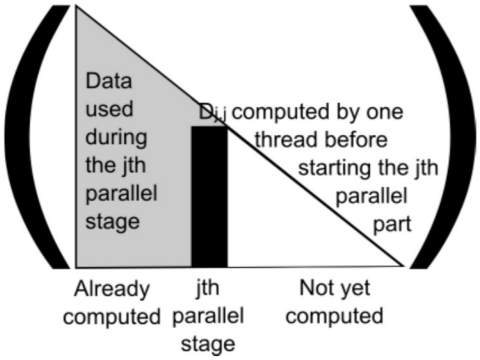


the parallel implementation of LDLt (cf. [1]) is performed as follows: for a fixed value of j, the different coefficients $\{L_{i,j}\}_{j+1\leq i\leq d}$ can be computed by at most $d-j$ independent threads. Thus, $\{L_{i,1}\}_{2\leq i\leq d}$ involves the biggest number of possible independent threads equal to $d-1$. In this collaborative version, we use the maximum $d-1$ threads $+1$ additional thread that is involved in the copy from global to shared and in the solution of the system after factorization. This makes d threads for the collaborative version and one of these threads is also involved in the computation $D_{j,j}$ which needs a synchronization before calculating $L_{i,j}$.

3. Implement a cuda code that computes all the pairs $(A_{t_k})_{k=1,\ldots,N-1}$ within a single kernel execution. Compare the values $A_{t_k}\cdot\psi(S^i_{t_k}, I^i_{t_k})$ to $F(t_k, S^i_{t_k}, I^i_{t_k})$ and estimate the regression error. (4 points + 2 points)

4 Merge big and batch sort small

The project begins with the implementation of the merge path algorithm introduced in [6], using a single CUDA block. Students are then required to extend this implementation to handle batched sorting of small arrays, followed by the merging of large arrays.

We start then with the merge path algorithm. Let A and B be two ordered arrays (increasing order), we want to merge them in an M sorted array. The merge of A and B is based on a path that starts at the top-left corner of the $|A|\times|B|$ grid and arrives at the down-right corner. The Sequential Merge Path is given by Algorithm 1 and an example is provided in the following figure.

Algorithm 1 Sequential Merge Path

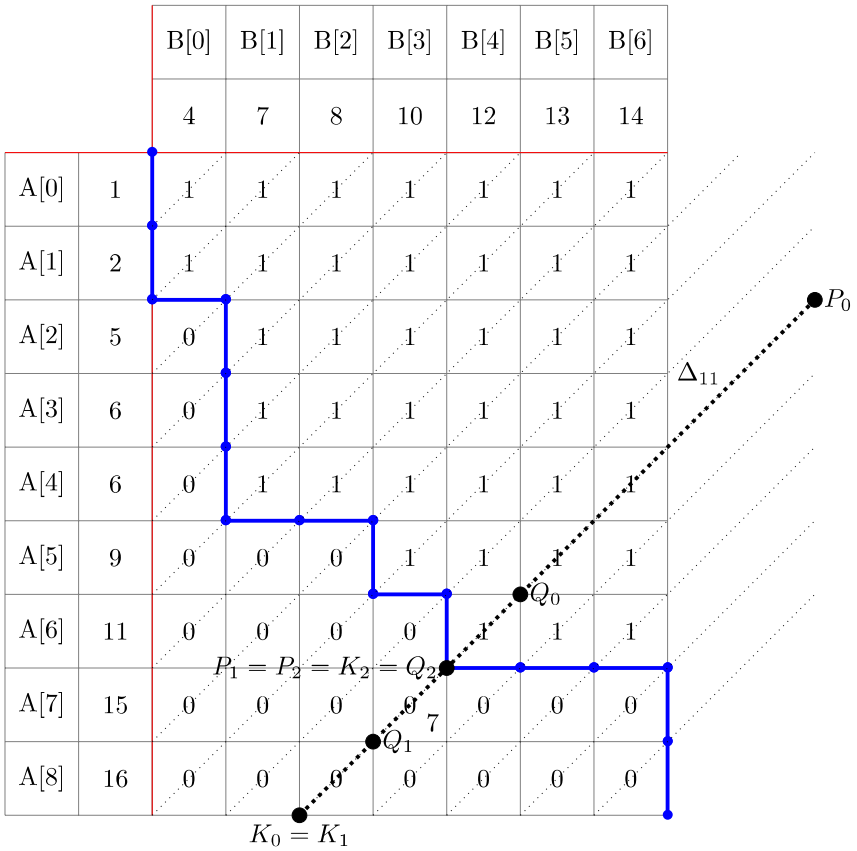
Require: A and B are two sorted arrays
Ensure: M is the merged array of A and B with $|M| = |A| + |B|$

```
procedure MergePath (A, B, M)
    j = 0 and i = 0
    while i + j < |M| do
        if i ≥ |A| then
            M[i+j]=B[j]
            j = j + 1                                   ▷ The path goes right
        else if j ≥ |B| or A[i] < B[j] then
            M[i+j]=A[i]                                 ▷ The path goes down
            i = i + 1
        else
            M[i+j]=B[j]
            j = j + 1                                   ▷ The path goes right
        end if
    end while
end procedure
```

Algorithm 2 Merge Path (Indices of n threads are integers from 0 to $n-1$)

Require: A and B are two sorted arrays
Ensure: M is the merged array of A and B with $|M| = |A| + |B|$

```
for each thread in parallel do
    i=index of the thread
    if i > |A| then
        K = (i − |A|, |A|)                          ▷ Low point of diagonal
        P = (|A|, i − |A|)                          ▷ High point of diagonal
    else
        K = (0, i)
        P = (i, 0)
    end if
    while True do
        offset = abs(K_y − P_y)/2
        Q = (K_x + offset, K_y − offset)
        if Q_y ≥ 0 and Q_x ≤ B and
          (Q_y = |A| or Q_x = 0 or A[Q_y] > B[Q_x − 1]) then
          if Q_x = |B| or Q_y = 0 or A[Q_y − 1] ≤ B[Q_x] then
              if Q_y < |A| and (Q_x = |B| or A[Q_y] ≤ B[Q_x]) then
                  M[i] = A[Q_y]                     ▷ Merge in M
              else
                  M[i] = B[Q_x]
              end if
              Break
          else
              K = (Q_x + 1, Q_y − 1)
          end if
        else
            P = (Q_x − 1, Q_y + 1)
        end if
    end while
end for
```



Each point of the grid has a coordinate $(i, j) \in [\![0, |A|]\!] \times [\![0, |B|]\!]$. The merge path starts from the point $(i, j) = (0, 0)$ on the left top corner of the grid. If $A[i] < B[j]$ the path goes down else it goes right. The array $[\![0, |A|-1]\!] \times [\![0, |B|-1]\!]$ of boolean values $A[i] < B[j]$ is not important in the algorithm. However, it shows clearly that the merge path is a frontier between ones and zeros.

To parallelize the algorithm, the grid has to be extended to the maximum size equal to $\max(|A|, |B|) \times \max(|A|, |B|)$. We denote K_0 and P_0 respectively the low point and the high point of the ascending diagonals Δ_k. On GPU, each thread $k \in [\![0, |A| + |B| - 1]\!]$ is responsible of one diagonal. It finds the intersection of the merge path and the diagonal Δ_k with a binary search described in Algorithm 2.

1. For $|A| + |B| \leq 1024$, write a kernel `mergeSmall_k` that merges A and B using only one block of threads. (8 points)

In this part, we assume that we have a large number $N(\geq 1e3)$ of arrays $\{A_i\}_{1 \leq i \leq N}$ and $\{B_i\}_{1 \leq i \leq N}$ with $|A_i| + |B_i| = d \leq 1024$ for each i. Using some changes on `mergeSmall_k`, we would like to write `mergeSmallBatch_k` that merges two by two, for each i, A_i and B_i.

Given a fixed common size $d \leq 1024$, `mergeSmallBatch_k` is launched using the syntax

```
mergeSmallBatch_k<<<numBlocks, threadsPerBlock>>>(...);
```

with `threadsPerBlock` is multiple of d but smaller than 1024 and `numBlocks` is an arbitrary sufficiently big number.

2.a) After figuring out why the indices
```
int Qt = threadIdx.x/d;
int tidx = threadIdx.x - (Qt*d);
int gbx = Qt + blockIdx.x*(blockDim.x/d);
```
are important in the definition of `mergeSmallBatch_k`, write the kernel `mergeSmallBatch_k` that batch merges two by two $\{A_i\}_{1 \leq i \leq N}$ and $\{B_i\}_{1 \leq i \leq N}$. Study the execution time with respect to d. (4 points)

2.b) Write the kernel `sortSmallBatch_k` that sorts many arrays $\{M_i\}_{1 \leq i \leq N}$ of size $d \leq 1024$. Study the execution time with respect to d. (4 points)

Actually, by using the index `idx = threadIdx.x + blockIdx.x * blockDim.x`, it is possible to replace the per-block thread index `threadIdx.x` used in question 1 to enable merging of large arrays across multiple blocks. However, this approach is suboptimal. Indeed, as mentioned in [6], merge path algorithm should be divided into 2 stages: partitioning stage and merging stage. The partitioning stage is important to propose an algorithm that involves various blocks.

3. For an input size $|A| + |B| = d$ large enough to fully engage the GPU's computational resources, design a parallel solution that merges arrays A and B by distributing the workload across multiple blocks through partitioning then study the execution time with respect to d. (4 points)

5 PDE simulation of bullet options

Assuming the Black & Scholes model, for time increments s and t with $0 \leq s < t \leq T$, the asset price process can be simulated using the following time induction formula:

$$S_t = S_s \exp\left((r - \sigma^2/2)(t - s) + \sigma\sqrt{t - s}G\right) \text{ and } S_0 = x_0 \text{ where}$$

- σ is the volatility assumed here equal to 0.2,
- r is the risk-free rate assumed here equal to 0.1,
- x_0 is the initial spot price of S at time 0, assumed here equal to 100